

Associação entre classes

CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Diagrama de classes UML

Linguagem de modelagem unificada – UML

- Linguagem (notação gráfica + semântica) para especificar, construir e documentar os artefatos dos sistemas
- Formas de uso para a UML
 - **Como rascunho** – diagramas incompletos e informais (geralmente escritos em um quadro branco) para explorar partes difíceis do problema
 - **Como projeto de software** – diagramas detalhados que podem ser usados para gerar código (engenharia direta) ou foram gerados para entender um código existente (engenharia reversa)

Perspectivas para aplicar a UML

Dado
valorDaFace

Perspectiva conceitual

Dado
- valorDaFace : int
+ obterValorDaFace() : int

Perspectiva de implementação

- **Perspectiva conceitual** – diagramas são interpretados como descrevendo coisas em uma situação no mundo real
- **Perspectiva de implementação** – diagramas descrevem implementações de software em uma tecnologia particular

Diagramas da UML

■ Diagramas estruturais

- Diagrama de classes
- Diagrama de objetos
- Diagrama de componentes
- Diagrama de implantação

■ Diagramas comportamentais

- Diagrama de caso de uso
- Diagrama de sequência
- Diagrama de colaboração
- Diagrama de atividades

Diagramas da UML

■ Diagramas estruturais

■ Diagrama de classes

- Diagrama de objetos

- Diagrama de componentes

- Diagrama de implantação

■ Diagramas comportamentais

- Diagrama de caso de uso

- Diagrama de sequência

- Diagrama de colaboração

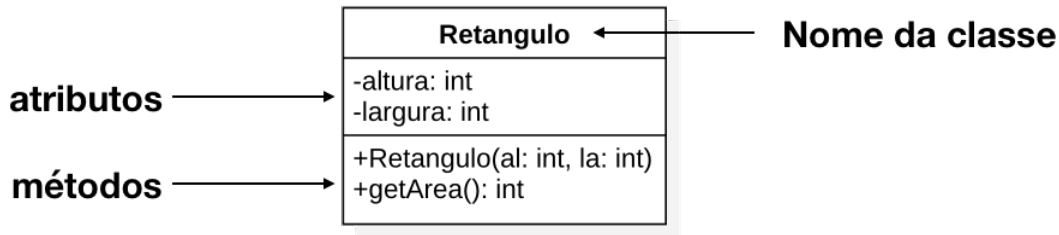
- Diagrama de atividades

Diagrama de classes

- **Classes** podem ser vistas como um **agrupamento de objetos de um mesmo tipo**, porém cada objeto é um exemplo único de um determinado grupo
- **Diagrama de classes** permite visualizar as classes que irão compor o sistema, com seus **atributos** e **métodos**, bem como em demonstrar como essas **classes se relacionam**

Um **diagrama de classes** é composto por suas **classes** e pelas **associações** existentes entre elas

Classe



- A representação de atributos e métodos é opcional
- Métodos triviais podem ser omitidos (p. ex: `getAltura()`)
- Modificadores de visibilidade (+, -, #)
- Membros estáticos ficam sublinhados

Ferramentas de modelagem UML

Veja mais em <https://emersonmello.me/ensino/poo/#modelagem-uml>

- **Mermaid** – <https://mermaid.live>
 - Modelagem é feita em texto para depois gerar a imagem do diagrama
- **PlantUML** – <https://plantuml.com>, <https://plantumleditor.com/webedit/>
 - Modelagem é feita em texto para depois gerar a imagem do diagrama
- **Draw.io** – <https://www.drawio.com>
 - Ferramenta gratuita de propósito geral para criar diagramas
 - É possível incluir diagramas feitos com Mermaid ou PlantUML
- **StarUML** – <http://staruml.io>
 - Aplicativo de modelagem UML multiplataforma (Windows, macOS, Linux)
 - Trata-se de uma ferramenta paga, mas é possível avaliar por 30 dias

Mermaid

<https://mermaid.live>

■ Documentação <https://mermaid.js.org/syntax/classDiagram.html>

```
```mermaid
classDiagram
 direction LR

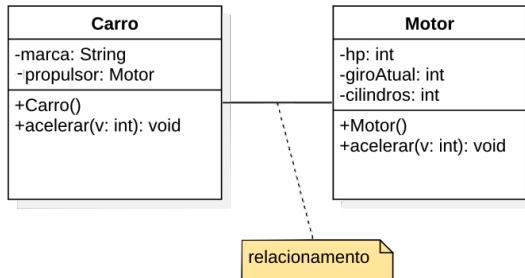
 class Retangulo{
 - altura : int
 - largura : int
 + Retangulo(int a, int l)
 + getArea() int
 }
```
```

Associação entre classes

Associação entre classes

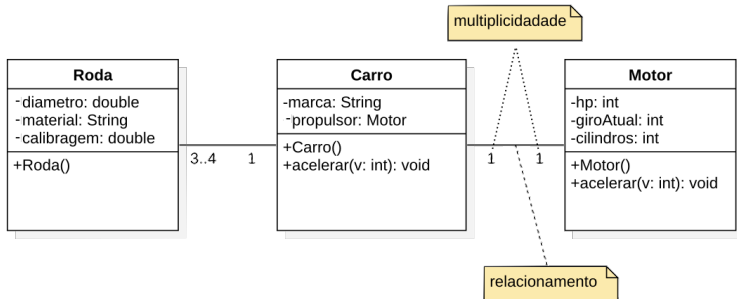
- Relacionamento entre classes que permite o compartilhamento de informação e colaboração para a execução de computação
- Descreve o vínculo entre objetos de uma ou mais classes
- A classe **Carro** possui um relacionamento com a classe **Motor**, pois um objeto da classe Carro contém 1 objeto da classe Motor

```
public class Carro{  
    private String marca;  
    private Motor propulsor;  
  
    public Carro(String m, Motor mo){  
        this.marca = m;  
        this.propulsor = mo;  
    }  
}
```



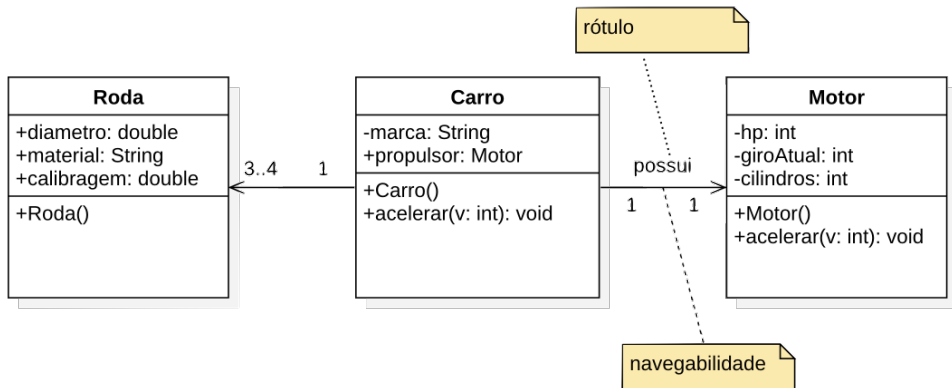
Associação bidirecional

| multiplicidade | |
|----------------|---------------|
| 0..* | Zero ou mais |
| 1..* | Um ou mais |
| * | Muitos |
| 1 | Exatamente um |
| 3..4 | De 3 a 4 |



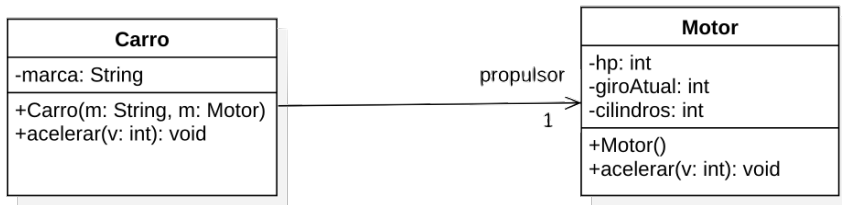
Associação unidirecional

- Indica o sentido em que as informações são transmitidas, isto é, indica o sentido em que os métodos poderão ser disparados
- Para acelerar, objeto da classe Carro invoca método do objeto da classe Motor

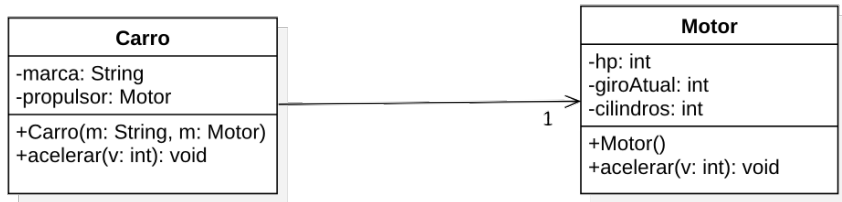


Associação: notação textual vs linha de associação

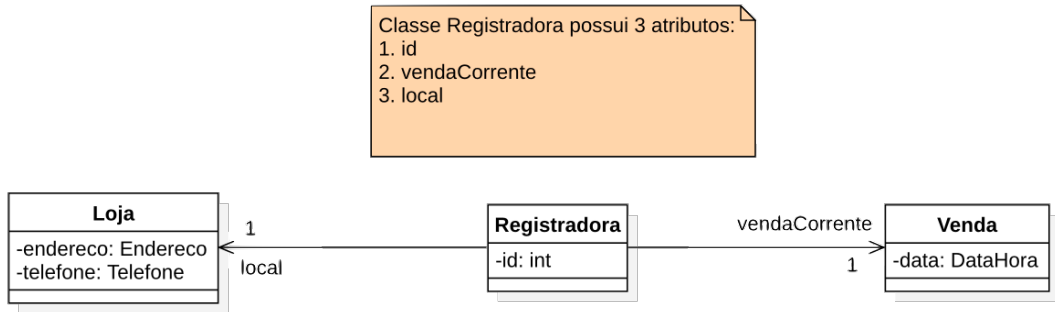
- Uso da notação de associação para indicar que Carro possui uma referência para uma instância de Motor



- Notação textual de atributo



Associação: notação textual vs linha de associação



Associação do tipo Agregação

- Para representar a relação entre o *objeto-todo* e *objetos-parte*
- Agregação ocorre quando uma **classe é uma coleção** ou contêiner **de outras classes**, porém o **ciclo de vida da classe contida não depende** fortemente da classe que a contém
- Quando o objeto contido faz sentido existir no sistema mesmo sem ser parte do objeto que o contém
- Serve para **indicar a obrigatoriedade de uma complementação das informações de um objeto-todo por seus objetos-parte**, quando esse for consultado

Associação do tipo Agregação

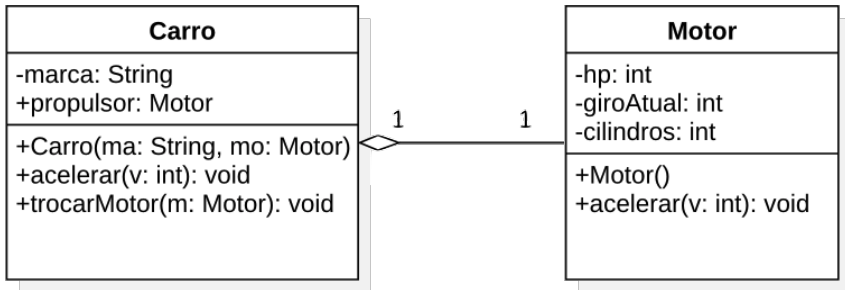
- Para representar a relação entre o *objeto-todo* e *objetos-parte*
- Agregação ocorre quando uma **classe é uma coleção** ou contêiner **de outras classes**, porém o **ciclo de vida da classe contida não depende** fortemente da classe que a contém
- Quando o objeto contido faz sentido existir no sistema mesmo sem ser parte do objeto que o contém
- Serve para **indicar a obrigatoriedade de uma complementação das informações de um objeto-todo por seus objetos-parte**, quando esse for consultado

Agregação pode ser substituída por uma associação simples

Um dos criadores da UML sugere em representar Agregação como uma associação simples

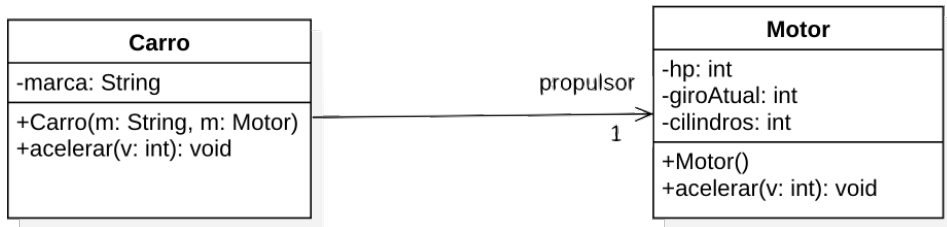
Associação do tipo Agregação

- Quando o objeto contido faz sentido existir no sistema mesmo sem ser parte do objeto que o contém
- Um **carro** possui um **motor**. Se o carro deixar de existir, o motor poderia ser colocado em outro carro



Associação do tipo Agregação

- Quando o objeto contido faz sentido existir no sistema mesmo sem ser parte do objeto que o contém
- Um **carro** possui um **motor**. Se o carro deixar de existir, o motor poderia ser colocado em outro carro

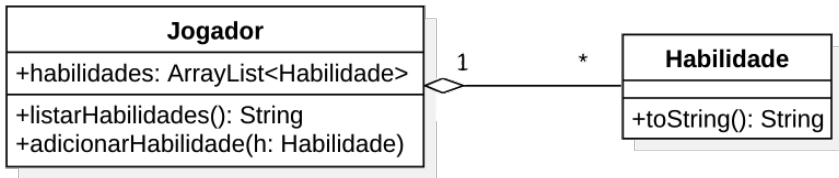


Associação do tipo Agregação

```
public class Carro{  
    private String marca;  
    private Motor propulsor;  
  
    public Carro(String m, Motor mo){  
        this.marca = m;  
        this.propulsor = mo;  
    }  
  
    public void acelerar(int valor){  
        this.propulsor.acelerar(valor);  
    }  
  
    public void trocarMotor(Motor mo){  
        this.propulsor = mo;  
    }  
}
```

Associação do tipo Agregação

- Serve para **indicar a obrigatoriedade de uma complementação das informações de um objeto-todo por seus objetos-parte**, quando esse for consultado



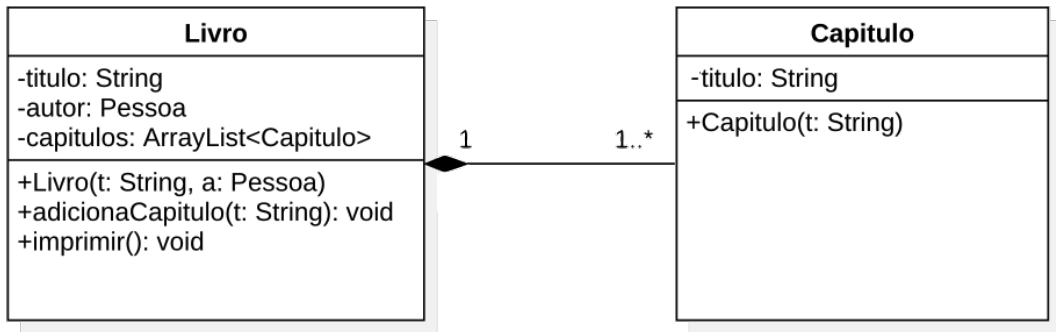
```
public String listarHabilidades(){
    StringBuilder sb = new StringBuilder();
    for(Habilidade item : habilidades){
        sb.append(item);
    }
    return sb.toString();
}
```

Associação do tipo Composição

- Vínculo mais forte entre o objeto-todo e o objeto-parte, demonstrando que os objeto-partes têm de estar associados a um único objeto-todo
- Objetos-parte não podem ser destruídos por um objeto diferente do objeto-todo ao qual estão relacionados
- Quando o **objeto contido** (objeto-parte) **não faz sentido existir** no sistema **se o objeto que o contém** (objeto-todo) **for excluído**

Associação do tipo Composição

- Um **livro** é composto por diversos **capítulos**. Se destruímos um **livro**, não faria mais sentido os capítulos existirem



Associação do tipo Composição

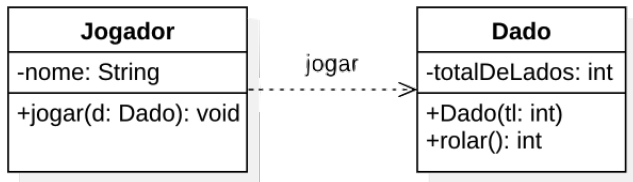
```
public class Livro{
    private String titulo;
    private ArrayList<Capitulo> capitulos;
    private Pessoa autor;

    public Livro(String t, Pessoa a){
        this.titulo = t;
        this.capitulos = new ArrayList<>();
        this.autor = a;
    }

    public void adicionaCapitulo(String titulo){
        this.capitulos.add(new Capitulo(titulo));
    }
}
```

Associação do tipo Dependência

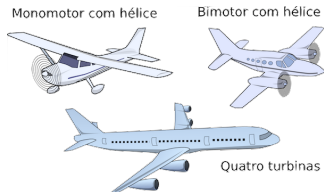
- Para indicar a dependência de uma classe em relação à outra
- Qualquer alteração na classe que é invocada pode resultar em uma quebra da classe que faz a invocação
- Uso temporário



```
public void jogar(Dado d){
    int res = d.rolar();
    ...
}
```

Exercício 1

Faça um diagrama de classes UML, nível conceitual, para um jogo



- Todo avião possui
 - Número máximo de tripulantes e número máximo de passageiros
 - Peso máximo de decolagem
 - Número máximo de combustível
- Cada avião pode possuir de 1 a 8 motores
 - Cada motor pode ser do tipo turbina ou hélice
 - Consumo de combustível por hora
- Cada motor pode ser ligado ou desligado

Modelando entidades e relaciona- mentos

Modelando classes de um sistema de informação

Entidades e relacionamentos

- **Entidade** é uma coisa ou objeto do mundo real que é **distinguível de todos os outros objetos**
 - Exemplos: Livro, Autor, Editora, Aluno, Curso
- **Relacionamento** é uma associação entre duas ou mais entidades
 - Exemplos: Livro **escrito por** Autor, Aluno **matriculado em** Curso
- **Atributos** são as propriedades que caracterizam uma entidade
 - Exemplos: Livro possui título, ISBN, ano de publicação, número de páginas, etc

Modelando classes de um sistema de informação

Entidades e relacionamentos

- **Entidade** é uma coisa ou objeto do mundo real que é **distinguível de todos os outros objetos**
 - Exemplos: Livro, Autor, Editora, Aluno, Curso
- **Relacionamento** é uma associação entre duas ou mais entidades
 - Exemplos: Livro **escrito por** Autor, Aluno **matriculado em** Curso
- **Atributos** são as propriedades que caracterizam uma entidade
 - Exemplos: Livro possui título, ISBN, ano de publicação, número de páginas, etc

As classes de um sistema de informação são modeladas como entidades, e os relacionamentos entre essas classes são modelados como associações entre as entidades

Modelando entidades e relacionamentos

Sistema para gestão de livros

| Livro |
|--|
| -isbn: String
-titulo: String
-idioma: String
-ano: int
-editora: String
-cidadeEditora: String
-autores: ArrayList<String>
-edicao: int
-paginas: int |

Modelando entidades e relacionamentos

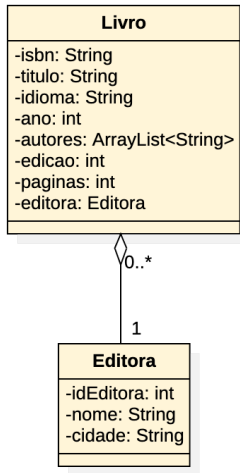
Sistema para gestão de livros

| Livro |
|--|
| -isbn: String
-titulo: String
-idioma: String
-ano: int
-editora: String
-cidadeEditora: String
-autores: ArrayList<String>
-edicao: int
-paginas: int |

- Uma Editora pode publicar vários livros
- cidadeEditora não é uma propriedade de Livro, mas sim de Editora

Modelando entidades e relacionamentos

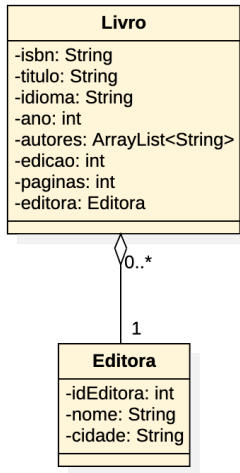
Sistema para gestão de livros



- Uma Editora pode publicar vários livros
- cidadeEditora não é uma propriedade de Livro, mas sim de Editora

Modelando entidades e relacionamentos

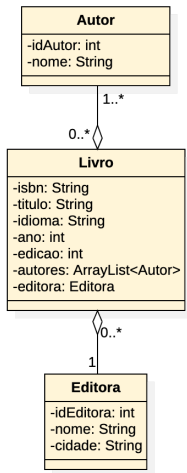
Sistema para gestão de livros



- Um **Livro** pode ter vários autores
- Um **Autor** pode ter escrito vários livros

Modelando entidades e relacionamentos

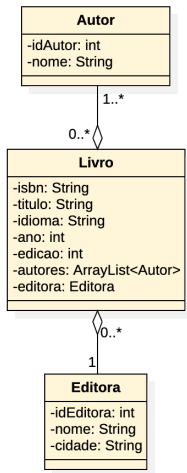
Sistema para gestão de livros



- Um **Livro** pode ter vários autores
- Um **Autor** pode ter escrito vários livros

Modelando entidades e relacionamentos

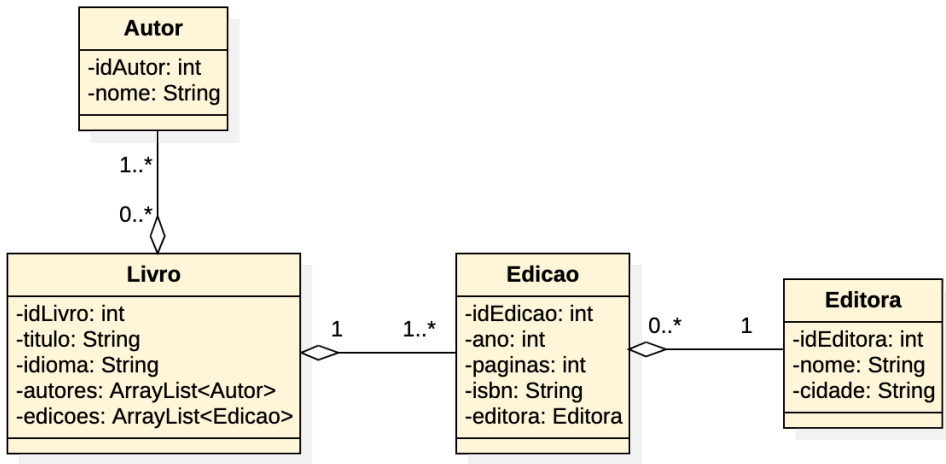
Sistema para gestão de livros



- Cada livro pode ter várias edições
- Diferentes edições de um mesmo livro podem ter diferentes ISBNs
- Cada edição tem um número de páginas e um ano de publicação
- Cada edição pode ser publicada por uma editora

Vamos praticar!

Ajuste o diagrama para contemplar as informações acima



Modelando entidades e relacionamentos

Sistema acadêmico

Aluno

- nome: String
- cpf: String
- dataNasc: LocalDate
- curso: String
- situacaoNoCurso: String
- matricula: String

Modelando entidades e relacionamentos

Sistema acadêmico

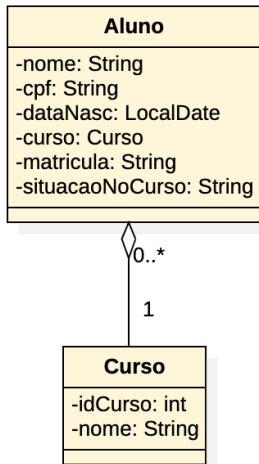
Aluno

-nome: String
-cpf: String
-dataNasc: LocalDate
-curso: String
-situacaoNoCurso: String
-matricula: String

- curso, que guarda o nome do curso, não é uma propriedade de Aluno, mas sim de Curso
- Um Curso pode ter vários alunos

Modelando entidades e relacionamentos

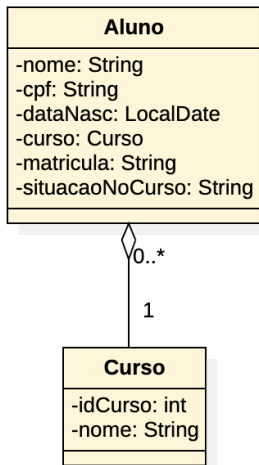
Sistema acadêmico



- curso, que guarda o nome do curso, não é uma propriedade de Aluno, mas sim de Curso
- Um Curso pode ter vários alunos

Modelando entidades e relacionamentos

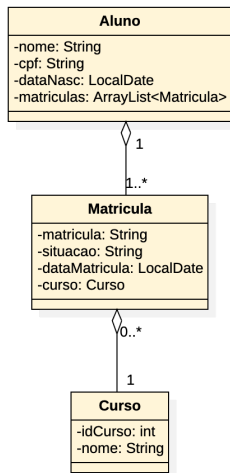
Sistema acadêmico



- É importante registrar a data da matrícula do aluno no curso
- Um aluno pode fazer vários cursos
- A situação do aluno no curso não é uma propriedade do aluno, mas sim de sua matrícula

Modelando entidades e relacionamentos

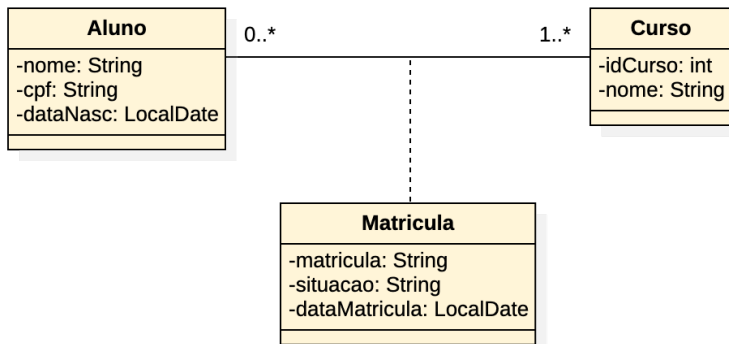
Sistema acadêmico



- É importante registrar a data da matrícula do aluno no curso
- Um aluno pode fazer vários cursos
- A situação do aluno no curso não é uma propriedade do aluno, mas sim de sua matrícula

Classe de associação

- A associação entre classes pode ser redefinida por uma nova classe a qual terá atributos próprios
- Adequada quando os atributos não pertencem a nenhuma das classes envolvidas na associação, mas à associação em si



Princípio da responsabilidade única

Princípio da responsabilidade única

*Single Responsibility Principle (SRP)*¹

"Ao escrever um módulo de software, você deve se lembrar de que ele deve ter uma e apenas uma razão para mudar, isto é, uma responsabilidade. Essas mudanças só podem ser solicitadas por uma única pessoa (Stakeholder)" –

Robert C. Martin, *Uncle Bob*

- **Cada classe deve ser responsável por uma pequena parte** da funcionalidade de um software
- A **responsabilidade** deve **estar completamente encapsulada** dentro da classe

¹ <https://blog.cleancoder.com/uncle-bob/2014/05/08/SingleResponsibilityPrinciple>

Separação de preocupações

Separation of Concerns (SoC)

- **Dividir** um sistema em **partes** que **cada uma é responsável por uma preocupação** diferente
- **Cada parte é desenvolvida, testada e mantida** separadamente
- Isso **facilita a manutenção, reutilização e evolução** do sistema

Exemplos

- Um sistema acadêmico pode ser dividido em módulos para gerenciar alunos, disciplinas, livros, etc
- Um sistema *web* pode ser dividido em módulos para gerenciar a interface com o usuário, a lógica de negócios e o armazenamento de dados

Princípio da responsabilidade única

Exemplo

Sistema para gestão de Agenda telefônica

- Usuário poderá:
 - Adicionar, Remover, Atualizar, Listar dados de um contato, Listar todos contatos
- Todo contato deve possuir:
 - Nome, Sobrenome, Data de nascimento, Telefone(s) e email(s)
- Todo telefone ou email deve possuir um rótulo de identificação (p. ex. celular, comercial, pessoal) e um valor (p. ex. +55048998761234)
 - Não deve ser permitido adicionar um email que não seja válido
- Ao listar os contatos, deve-se exibir o nome completo, data de nascimento, telefone(s) e email(s) de cada contato
 - Os telefones e emails devem ser exibidos com o rótulo de identificação
 - Telefones devem ser exibidos formatados, no formato internacional: Ex: +55048998761234 → +55 (48) 9 9876-1234

Sistema para gestão de Agenda telefônica

Seguindo o princípio da responsabilidade única, como você modelaria o sistema?

Vamos praticar!

Faça um diagrama de classes UML com **somente os nomes das classes** que serão necessárias, **bem como suas associações**

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- App e Contato?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- App e Contato?
 - App está assumindo a lógica de gerenciar contatos
 - A classe App deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- App e Contato?
 - App está assumindo a lógica de gerenciar contatos
 - A classe App deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- App, Agenda e Contato?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- App e Contato?
 - App está assumindo a lógica de gerenciar contatos
 - A classe App deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- App, Agenda e Contato?
 - Quem será responsável por garantir que os emails são válidos?
 - A classe Contato deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- App e Contato?
 - App está assumindo a lógica de gerenciar contatos
 - A classe App deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- App, Agenda e Contato?
 - Quem será responsável por garantir que os emails são válidos?
 - A classe Contato deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails
- App, Agenda, Contato e ColecaoEmail?

Sistema para gestão de Agenda telefônica

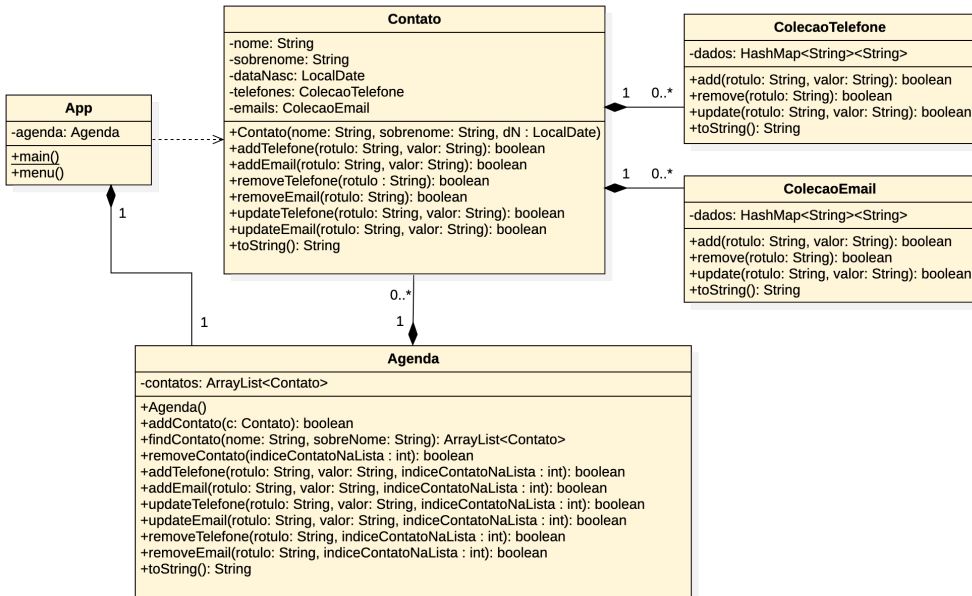
Quantas classes serão necessárias?

- App e Contato?
 - App está assumindo a lógica de gerenciar contatos
 - A classe App deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- App, Agenda e Contato?
 - Quem será responsável por garantir que os emails são válidos?
 - A classe Contato deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails
- App, Agenda, Contato e ColecaoEmail?
 - Quem será responsável por formatar os telefones?

Sistema para gestão de Agenda telefônica

Quantas classes serão necessárias?

- App e Contato?
 - App está assumindo a lógica de gerenciar contatos
 - A classe App deve ser responsável somente pela interface com o usuário, não deve conter lógica de negócio
- App, Agenda e Contato?
 - Quem será responsável por garantir que os emails são válidos?
 - A classe Contato deve ser responsável somente por armazenar os dados do contato, não deve conter lógica para validar emails
- App, Agenda, Contato e ColecaoEmail?
 - Quem será responsável por formatar os telefones?
- App, Agenda, Contato, ColecaoEmail e ColecaoTelefone



Material de apoio

Classes `java.time.LocalDate` e `java.time.LocalDateTime`

■ `java.time.LocalDate`

```
import java.time.Month;
//.....
// Dia de hoje
LocalDate hoje = LocalDate.now();
// Criando objeto com uma data específica
LocalDate dataEspecifica = LocalDate.of(2020, Month.JANUARY, 10);
```

■ `java.time.LocalDateTime`

```
import java.time.format.DateTimeFormatter;
//.....
String dhF;

LocalDateTime dataHora = LocalDateTime.of(2019, 04, 22, 13, 30, 00);
dhF = dataHora.format(DateTimeFormatter.ofPattern("yyyyMMdd'T'HHmmss"));
System.out.printf(dhF); // Será impresso: 20190422T133000

// para obter a data e hora no momento que essa linha foi executada
LocalDateTime agora = LocalDateTime.now();
```

Expressão regular (*regex*)

| Metacaractere | Descrição |
|---------------|--|
| . | Casa com qualquer caractere |
| [] | Lista de caracteres. Ex: <code>n[aã]</code> o casa com <i>não</i> e <i>nao</i> . É possível definir intervalos <code>[0-2]</code> casa com 0, 1 ou 2 |
| ^ | Casa com o começo da cadeia de caracteres |
| \$ | Casa com o final da cadeia de caracteres |
| * | Casa o elemento precedente zero ou mais vezes. Ex: <code>ab*c</code> casa com <code>ac</code> , <code>abc</code> , <code>abbbbc</code> |
| + | Casa o elemento precedente um ou mais vezes |
| ? | Torna o elemento precedente opcional |
| ! | Negação. Ex: <code>n[!a]</code> casa com <code>n</code> , mas não com <code>na</code> , nem <code>não</code> |
| \ | Para escapar outros metacaracteres. |
| {n,m} | De <code>n</code> até <code>m</code> . Ex: <code>2</code> , pelo menos 2 caracteres |
| () | Grupo. Ex: <code>(www\.)?ifsc.edu.br</code> casa com <code>www.ifsc.edu.br</code> e <code>ifsc.edu.br</code> |
| \w | Corresponde a <code>[a-zA-Z_0-9]</code> |

Expressão regular (regex)

Exemplos em Java

```
// Para garantir que somente números sejam digitados
String eR = "[0-9]+$";
String numero = "1234567890";
System.out.println("Número válido? " + numero.matches(eR));

// Para garantir que somente letras sejam digitadas
String eR = "[a-zA-Z]+$";
String letras = "abcde";
System.out.println("Letras válidas? " + letras.matches(eR));

// Para garantir que somente letras e números sejam digitados
String eR = "[a-zA-Z0-9]+$";
String alfanumerico = "abc123";
System.out.println("Alfanumérico válido? " + alfanumerico.matches(eR));

// Para não permitir caracteres repetidos e que tenha pelo menos 8 caracteres
String eR = "(?!.*(.)\\1{2,})[a-zA-Z0-9]{8,}$";
String senha = "abc12345";
System.out.println("Senha válida? " + senha.matches(eR));
```

Expressão regular (*regex*)

Exemplo de validação de email

```
String eR = "^([\\w-\\+]+(\\. [\\w]+)*)@[\\w-]+(\\. [\\w]+)*(\\. [a-z]{2,})$";  
  
String email = "meu.email@dominio.com.br";  
  
System.out.println("Email válido? " + email.matches(eR));
```

- <https://docs.oracle.com/javase/tutorial/essential/regex/>
- <https://docs.oracle.com/javase/8/docs/api/java/util/regex/Pattern.html>

Classe MaskFormatter

- A classe `MaskFormatter` é usada para formatar e editar Strings
- A máscara indica quais são os caracteres válidos que podem estar contidos na String

| | |
|---|---|
| # | Qualquer número |
| U | Qualquer caractere e todos serão convertidos para maiúsculo |
| L | Qualquer caractere e todos serão convertidos para minúsculo |
| A | Qualquer caractere ou número |
| ? | Qualquer caractere |
| * | Qualquer coisa |
| H | Qualquer hexadecimal (0-9, a-f ou A-F) |

```
MaskFormatter mask = new MaskFormatter("(##) #####-####");
```

- O construtor dispara uma exceção do tipo `ParseException`

Exemplo com classe MaskFormatter

```
public String formata(String mascara, String valor){
    MaskFormatter mask = null;
    String resultado = "";

    try {
        mask = new MaskFormatter(mascara);
        mask.setValueContainsLiteralCharacters(false);
        mask.setPlaceholderCharacter('_');
        resultado = mask.valueToString(valor);
    } catch (ParseException e) {
        e.printStackTrace();
    }

    return resultado;
}

public static void main(String[] args) {
    App p = new App();
    System.out.println(p.formata("(##) #####-####", "48998765432"));
}
```

Leitura obrigatória



Craig Larman

Utilizando UML e padrões

3a. Edição - Editora Bookman, 2007

- Capítulos 16, seções: 16.1 – 16.9, 16.10, 16.13



Gilleanes T.A. Guedes

UML2 – uma abordagem prática

2a. Edição - Editora Novatec, 2011

- Capítulos 4, seções: 4.1–4.2.2, 4.2.4, 4.2.5, 4.2.8

Aulas baseadas em



Craig Larman

Utilizando UML e padrões

3a. Edição - Editora Bookman, 2007